

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №5

Тема: Патерни проектування

Виконав:
студент групи ІА-34
Єресько І.С
Перевірив:
Мягкий М.Ю

Короткі теоретичні відомості

5.2.1. Шаблон «Adapter» (Адаптер)

Призначення патерну

Патерн «Адаптер» слугує для узгодження інтерфейсів двох несумісних між собою об'єктів. Його застосовують тоді, коли необхідно використовувати сторонній або застарілий компонент із інтерфейсом, який не підходить до решти системи.

Наприклад, існують різні бібліотеки для взаємодії з принтерами. Вони виконують схожі функції, але мають різні способи виклику методів. Створивши єдиний універсальний інтерфейс та реалізувавши адаптери для кожної бібліотеки, можна забезпечити роботу програми через один набір методів, не залежачи від специфіки конкретних бібліотек. Часто адаптери також називають обгортками (wrapper).

Проблема

Під час створення аудіоплеєра виявляється, що для різних форматів аудіо краще використовувати різні компоненти. Через це код переповнюється умовами: якщо формат один - викликаємо певний метод, якщо інший - використовуємо інший компонент, і так далі.

Коли з'являється необхідність додати новий формат, доводиться змінювати вже написану логіку, що підвищує ризик виникнення нових помилок.

Рішення

Патерн Adapter дозволяє стандартизувати процес взаємодії з різними компонентами. Для цього створюється інтерфейс IPlayer - загальний контракт для відтворення аудіо.

Для кожного компонента пишеться окремий адаптер, який зберігає посилання на конкретний об'єкт і реалізує загальні методи, перетворюючи їх на виклики внутрішнього компоненту.

Основний код працює лише з інтерфейсом IPlayer, не знаючи деталей реалізації.

Додавання нового формату зводиться до створення нового адаптера, без змін у клієнтському коді.

Переваги та недоліки

Переваги:

Дозволяє ізолювати складну логіку взаємодії з різними компонентами.

Спрощує розширення системи - достатньо додати новий адаптер.

Недолік:

– Збільшується кількість класів, але код стає зрозумілішим та структурованішим.

5.2.2. Шаблон «Builder»

Призначення патерну

Патерн «Builder» застосовується для відокремлення процесу створення складного об'єкта від того, як саме він виглядає. Це корисно, коли об'єкт має багато етапів формування або кілька можливих варіантів побудови. Прикладом може бути процес формування відповіді web-сервера або конвертація документа між форматами.

Проблема

Формування відповіді web-сервера складається з кількох кроків: додавання заголовків, визначення статусу, запис вмісту, налаштування типу контенту та інше.

Усе це з часом перетворюється на громіздку послідовність викликів, яку важко змінювати і підтримувати.

Рішення

Патерн Builder дозволяє рознести різні етапи побудови на окремі методи, що робить процес формування об'єкта більш гнучким.

Можна змінювати внутрішню логіку, не порушуючи структуру створення кінцевого продукту. Це також спрощує можливість створювати різні варіанти одного й того ж об'єкта.

Переваги та недоліки

Переваги:

Один і той самий код побудови можна використовувати для створення різних об'єктів.

Легко контролювати та змінювати етапи створення складного продукту.

Недолік:

– Клієнт може залежати від конкретного будівельника, якщо останній не надає методу повернення результату.

5.2.3. Шаблон «Command» (Команда)

Призначення патерну

Патерн Command перетворює будь-яку операцію у окремий об'єкт. Це дозволяє зберігати інформацію про дію, логувати її, відмінити та повторювати. Підхід використовується там, де система має багато команд і важливо забезпечити їх розширення.

Проблема

У складних програмах із меню, кнопками та контекстними меню одна й та сама дія викликається з різних місць. Зазвичай логіку доводиться дублювати для кожного елемента інтерфейсу.

Підтримка й тестування такої логіки ускладнюється, оскільки потрібно емулювати взаємодію користувача.

Рішення

Винесення дії у окремий клас-команду дозволяє повністю відокремити UI від логіки виконання.

Команди реалізують спільний інтерфейс, тому меню, кнопки й інші елементи просто викликають метод Execute() не знаючи деталей реалізації.

Команди легко тестуються, оскільки їх можна викликати програмно без UI.

Також просто реалізувати механізм Enable/Disable та системи відміни.

Переваги та недоліки

Переваги:

Клієнтський код не залежить від реалізації команд.

Легко додавати можливість скасування або повторення.

Команди можна логувати та виконувати повторно.

Проста можливість розширення системи.

Недолік:

– Команд стає багато, що збільшує кількість класів.

5.2.4. Шаблон «Chain of Responsibility»

Призначення патерну

Патерн «Ланцюжок відповідальності» дозволяє передавати запит уздовж послідовності об'єктів, поки один з них не обробить його. Він часто нагадує реальні робочі процеси, коли документ переходить від одного співробітника до іншого.

Проблема

У складних формах з великою кількістю вкладених елементів потрібно формувати контекстне меню залежно від того, на якому компоненті стався клік.

Програміст змушений писати громіздкі перевірки, визначати, чи містить один компонент інший, і додавати відповідні пункти меню. З часом логіка перетворюється на хаотичний код, в якому легко виникають помилки.

Рішення

Кожен компонент реалізує інтерфейс із методом UpdateContextMenu().

Компонент, отримавши запит, додає свої пункти меню й передає обробку своєму «батьківському» елементу.

Таким чином формується ланцюг, у якому кожен компонент додає тільки те, що належить йому.

При зміні структури форми змінюються лише зв'язки між компонентами, а логіку переписувати не потрібно.

Переваги та недоліки

Переваги:

Клієнт не залежить від конкретних обробників.

Легко додавати або видаляти об'єкти з ланцюга.

Недоліки:

– Запит може так і не бути обробленим, якщо жоден елемент не відповідає за нього.

5.2.5. Шаблон «Prototype»

Призначення патерну

Патерн «Прототип» дозволяє створювати нові об'єкти шляхом копіювання існуючих замість створення їх «з нуля». Кожен клас-прототип повинен реалізувати метод клонування.

Цей підхід підходить для ситуацій, коли структура кінцевого об'єкта відома заздалегідь, а створення нового екземпляра значно простіше виконати копіюванням.

Проблема

У редакторі ігрових рівнів є велика кількість можливих елементів (стіни, сходи, декорації тощо), які належать до спільного базового класу.

Якщо для кожного елемента створювати окрему кнопку, у програмі виникне дублювання ієрархій. Додавання нового типу об'єкта вимагатиме створення нового типу кнопки.

Рішення

Кнопки зберігають посилання не на конкретний клас, а на базовий GameObject - прототип.

Під час додавання елемента на сцену відбувається не створення нового об'єкта, а клонування прототипу.

Це дозволяє уникнути дублювання логіки та не вимагати зміни коду при появі нових видів об'єктів.

Переваги та недоліки

Переваги:

Значно швидше створювати складні об'єкти через копіювання.

Можна отримувати різні варіації елементів без збільшення ієрархії класів.

Кожна копія може налаштовуватися окремо, не змінюючи прототип.

Недоліки:

- Реалізувати глибоке копіювання складних структур доволі важко.
- Надмірне використання може призвести до заплутаного коду.

Тема: Патерни проектування.

Мета: Вивчити структуру шаблонів «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та навчитися застосовувати їх в реалізації програмної системи.

Хід роботи

8. Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server)

Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або одночасно шляхом розділення вікна)

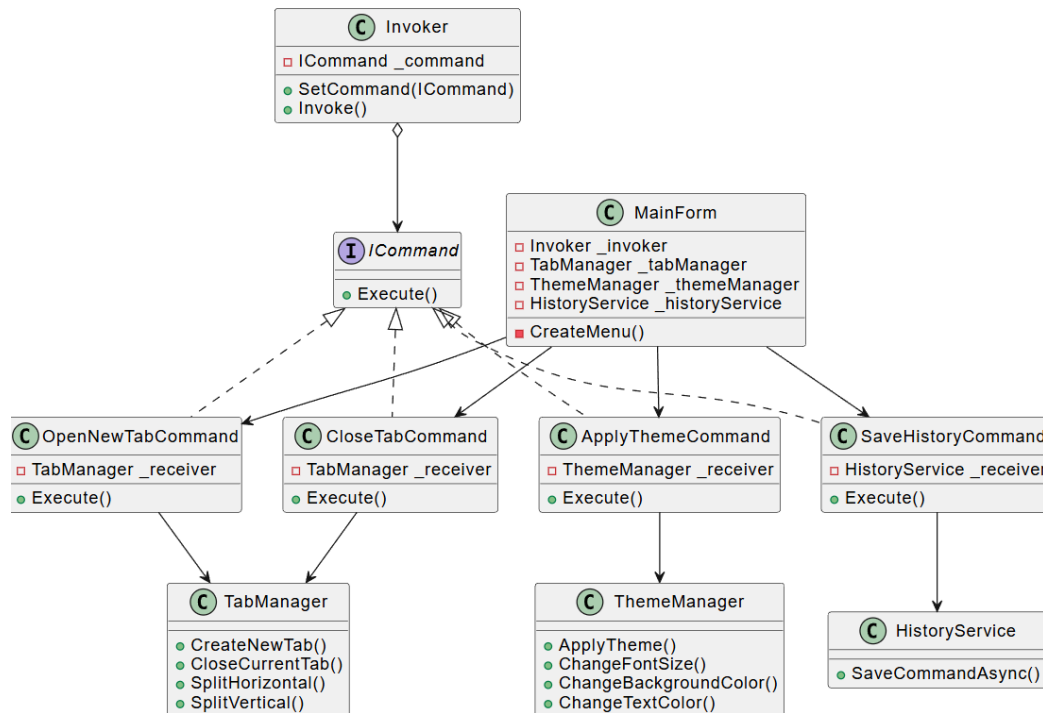


Рис 1 :Command class diagram

[ICommand](#)

[CommandInvoker](#)

[OpenNewTabCommand](#)

[CloseTabCommand](#)

[ApplyDarkThemeCommand](#)

[ApplyLightThemeCommand](#)

[SaveHistoryCommand](#)

Ця діаграма демонструє реалізацію патерна **Command** у застосунку, де логіка виконання дій відокремлена від місця, де ці дії ініціюються. У центрі знаходиться інтерфейс **ICommand**, який визначає єдиний метод `Execute()` — спільний для всіх команд системи. Він виступає абстракцією, що дозволяє `MainForm` працювати з будь-якою командою, не знаючи деталей її реалізації.

У верхній частині розташований клас **Invoker**, який зберігає поточну команду та викликає її виконання через метод `Invoke()`. Саме через `Invoker` `MainForm` ініціює дії користувача. `MainForm` виступає ініціатором команд та містить посилання на `Invoker`, а також сервіси, що будуть отримувачами команд: `TabManager`, `ThemeManager` та `HistoryService`. У своєму методі `CreateMenu()` головна форма створює відповідні об'єкти команд і передає їх `Invoker`-у для виконання.

Кожна конкретна команда реалізує інтерфейс `ICommand` та виконує певну дію над відповідним отримувачем (receiver). **OpenNewTabCommand** і **CloseTabCommand** працюють із `TabManager`: одна створює нову вкладку, інша закриває поточну. **ApplyThemeCommand** використовує `ThemeManager` для застосування нового оформлення інтерфейсу. **SaveHistoryCommand** звертається до `HistoryService` для збереження команд користувача.

У нижній частині діаграми представлені класи-отримувачі: `TabManager`, `ThemeManager` та `HistoryService`. Кожен із них має методи, які безпосередньо виконують роботу — створюють вкладку, змінюють тему чи записують історію в базу. Команди не містять власної логіки — вони лише делегують завдання своїм отримувачам.

Таким чином діаграма ілюструє повне розділення відповідальностей: `MainForm` лише ініціює дії, `Invoker` викликає команди, самі команди знають лише одного отримувача, а вся реальна логіка міститься у відповідних сервісах. Така структура забезпечує модульність, розширюваність та можливість легко додавати нові команди без зміни існуючих компонентів.

Висновок

У ході виконання роботи було детально досліджено ключові патерни проектування - `Adapter`, `Builder`, `Command`, `Chain of Responsibility` та `Prototype`. Вивчення їхньої структури, принципів взаємодії класів та практичних сценаріїв використання дозволило глибше зрозуміти підхід до побудови гнучких та масштабованих програмних систем. Кожен шаблон вирішує специфічну проблему: «Адаптер» забезпечує сумісність інтерфейсів, «Будівельник» дозволяє створювати складні об'єкти покроково, «Команда» інкапсулює запити як окремі об'єкти, «Ланцюжок

відповідальності» забезпечує послідовну обробку запитів різними обробниками, а «Прототип» дає можливість швидко створювати копії об'єктів.

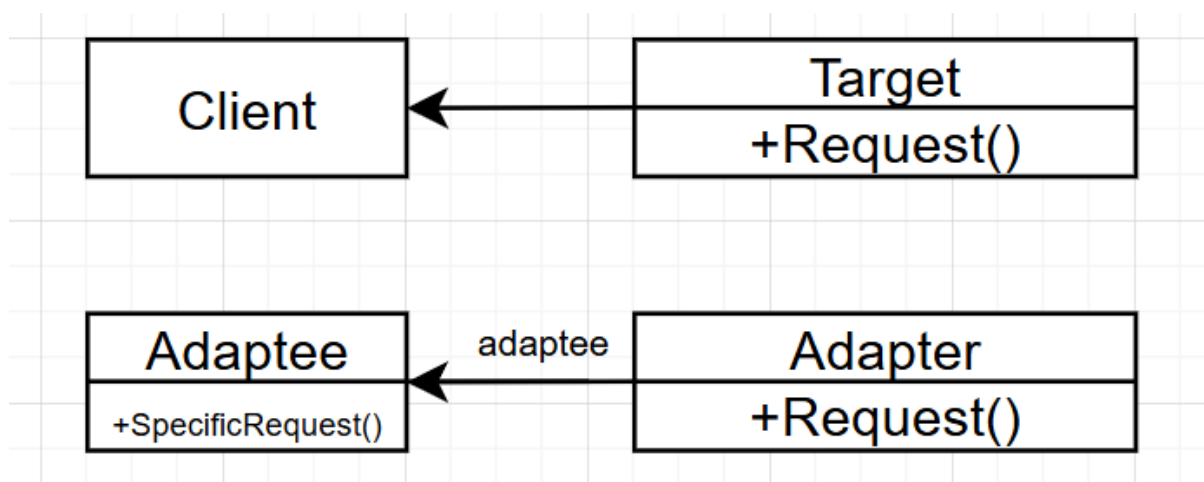
Застосування цих патернів у програмуванні підвищує модульність, полегшує розширення системи та зменшує зв'язність між компонентами. У результаті розробник отримує інструментарій, який робить архітектуру програмних рішень більш гнучкою, зрозумілою та легко підтримуваною. Отже, вивчення і практичне застосування шаблонів проектування є важливим етапом формування професійних навичок у сфері розробки програмного забезпечення

Контрольні запитання

1. Яке призначення шаблону «Адаптер»?

Шаблон «Адаптер» призначений для узгодження інтерфейсів двох несумісних між собою класів. Він дозволяє використовувати існуючий клас із новим інтерфейсом без зміни його внутрішньої реалізації. Адаптер виступає прошарком, який перетворює виклики клієнта у формат, що відповідає інтерфейсу адаптованого класу. Це дозволяє повторно використовувати вже готові компоненти та інтегрувати сторонні бібліотеки, не змінюючи їхній код.

2. Нарисуйте структуру шаблону «Адаптер».



3. Які класи входять у шаблон «Адаптер», та яка між ними взаємодія?

У шаблон входять чотири класи: **Client**, **Target**, **Adapter** та **Adaptee**. Клієнт взаємодіє виключно з Target-інтерфейсом. Adapter реалізує цей інтерфейс та всередині містить посилання на Adaptee. Будь-який виклик Client → Adapter перенаправляється до Adaptee з відповідним перетворенням параметрів чи формату. Таким чином клієнт не підозрює, що фактично працює з іншим класом.

4. Різниця між адаптером на рівні об'єктів та на рівні класів

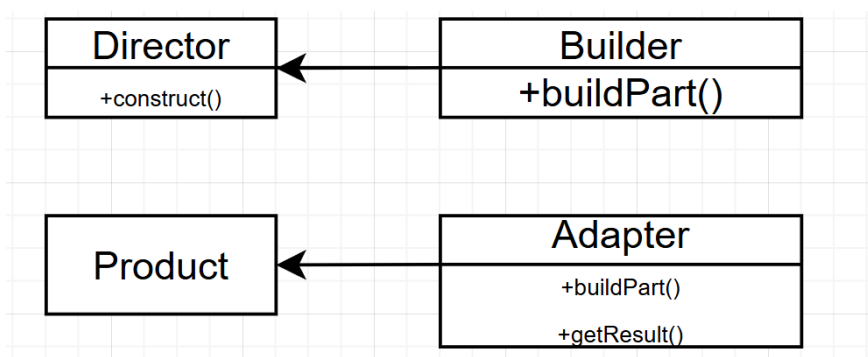
Адаптер на рівні об'єктів використовує композицію - Adapter містить всередині екземпляр Adaptee. Це гнучкіше рішення, оскільки дозволяє підмінювати адаптований об'єкт під час роботи.

Адаптер на рівні класів застосовує наслідування - Adapter успадковує інтерфейс Target та реалізацію Adaptee. Такий спосіб можливий лише в мовах, що підтримують множинне наслідування, і має меншу гнучкість, оскільки прив'язує адаптер до конкретного класу.

5. Яке призначення шаблону «Будівельник»?

Шаблон «Будівельник» використовується для покрокового створення складних об'єктів. Він дозволяє конструювати об'єкти з різними конфігураціями, не змінюючи їхніх класів. Окремі етапи створення приховані у Builder-класах, а процесом керує Director. Головна перевага - можливість створювати різні варіації об'єктів, використовуючи один і той самий сценарій побудови.

6. Нарисуйте структуру шаблону «Будівельник».



7. Які класи входять у шаблон «Будівельник», та яка між ними взаємодія?

Director задає порядок виконання кроків, але не знає деталей реалізації. Builder описує загальні кроки створення об'єкта. ConcreteBuilder реалізує їх, формуючи об'єкт різних конфігурацій. Наприкінці роботи ConcreteBuilder повертає готовий Product. Така взаємодія відокремлює алгоритм побудови від деталей реалізації.

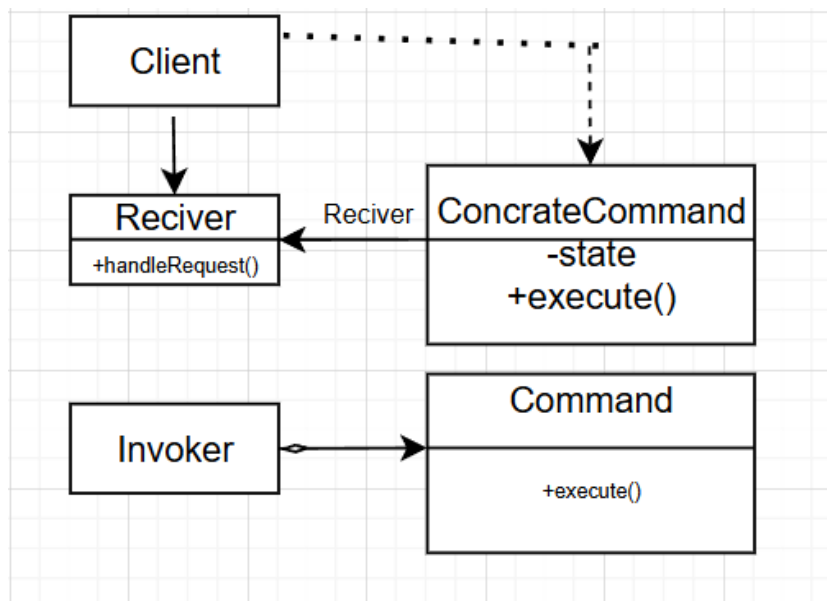
8. У яких випадках варто застосовувати шаблон «Будівельник»?

Builder застосовують тоді, коли потрібно створювати складні об'єкти, що мають багато параметрів або різні варіанти конфігурації. Особливо корисний, коли процес створення має фіксовану послідовність кроків, але різні реалізації цих кроків приводять до різних результатів. Підходить для побудови документів, UI-компонентів, складних моделей чи конфігурацій.

9. Яке призначення шаблону «Команда»?

Шаблон «Команда» призначений для інкапсуляції операції у вигляді окремого об'єкта. Це дозволяє зберігати, передавати, логувати та відмінити команди. Клас інкапсулює як дію, так і отримувача (receiver), що її виконує. Завдяки цьому можна реалізовувати історію операцій, черги команд, макрокоманди та інші високорівневі сценарії.

10. Нарисуйте структуру шаблону «Команда».



11. Які класи входять у шаблон «Команда», та яка між ними взаємодія?

Client створює ConcreteCommand і передає Invoker-у. Команда знає свого Receiver і при виконанні передає йому запит. Invoker викликає метод execute() у команди, не розуміючи, що саме вона робить. Receiver містить фактичну логіку виконання операції. Такий розподіл забезпечує високий рівень гнучкості.

12. Розкажіть як працює шаблон «Команда».

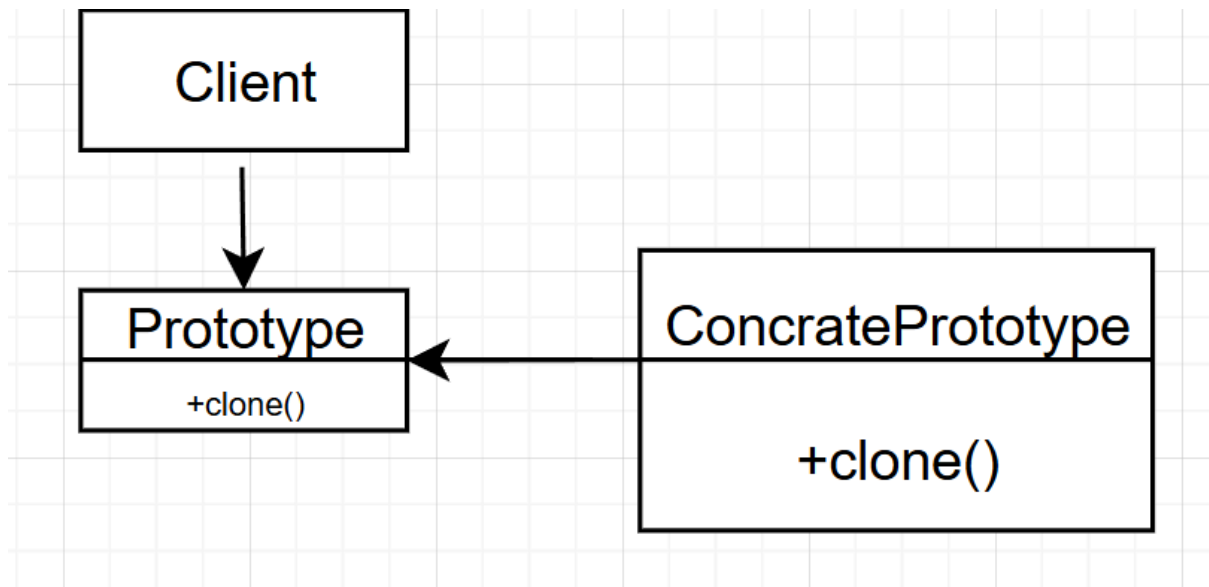
Шаблон працює так: замість того, щоб викликати метод безпосередньо, клієнт створює об'єкт команди, який містить посилання на отримувача та описує дію. Інвокер отримує команду і викликає її метод execute(). Команда делегує дію реальному об'єкту Receiver. Завдяки цьому централізується керування командами - їх можна ставити у чергу, відкладати, скасовувати і комбінувати.

13. Яке призначення шаблону «Прототип»?

Шаблон «Прототип» дозволяє створювати нові об'єкти шляхом копіювання існуючих екземплярів. Він корисний, коли створення об'єкта є дорогим або складним, а копіювати його значно простіше. Прототип дає

можливість динамічно генерувати нові об'єкти без залежності від їхніх конкретних класів.

14. Нарисуйте структуру шаблону «Прототип».



15. Які класи входять у шаблон «Прототип», та яка між ними взаємодія?

Prototype задає контракт копіювання. ConcretePrototype реалізує метод `clone()`, що повертає новий об'єкт з аналогічним станом. Client зберігає посилання на об'єкт-прототип і створює нові об'єкти без використання оператора `new` - замість цього він викликає `clone()`, що дозволяє створювати об'єкти динамічно.

16. Які приклади використання шаблону «Ланцюжок відповідальності»?

Шаблон використовують у системах, де запит має проходити через послідовність обробників і кожен з них може або обробити запит, або передати далі. Приклади: обробка подій у GUI, системи логування, фільтри HTTP-запитів, механізми валідації, маршрутизація повідомлень, а також побудова middleware у веб-фреймворках. Це дозволяє створити гнучку, динамічну і розширювану систему обробки даних.